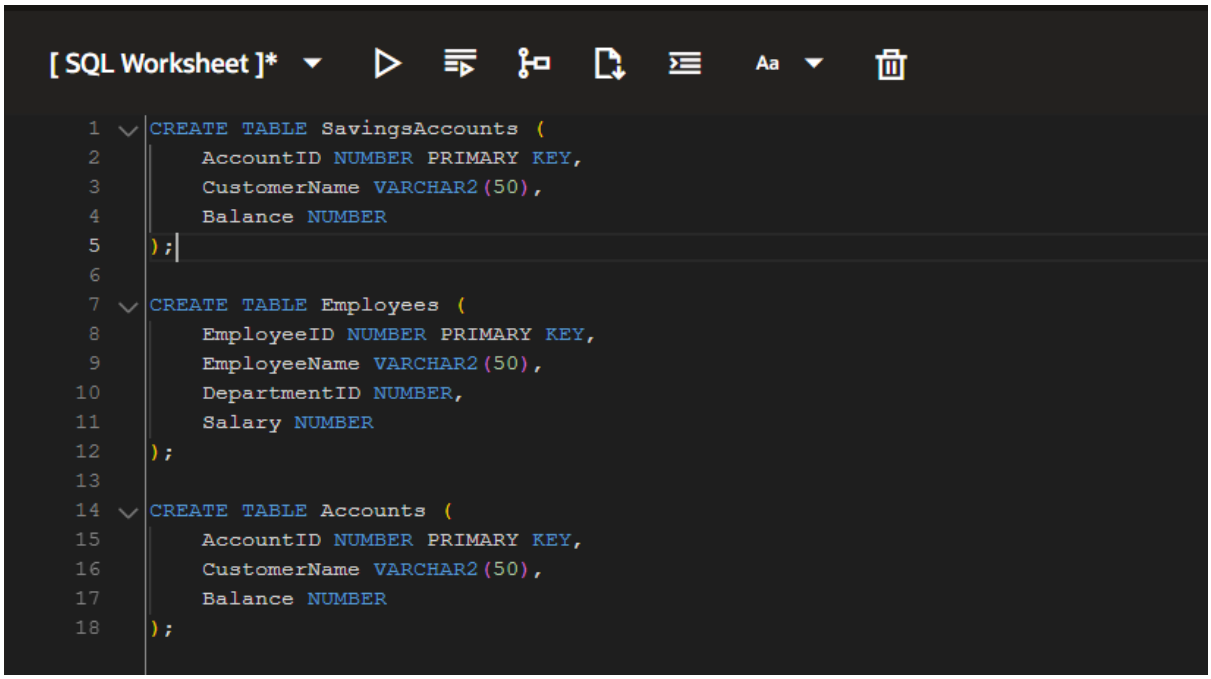
PLSQL_Exercises

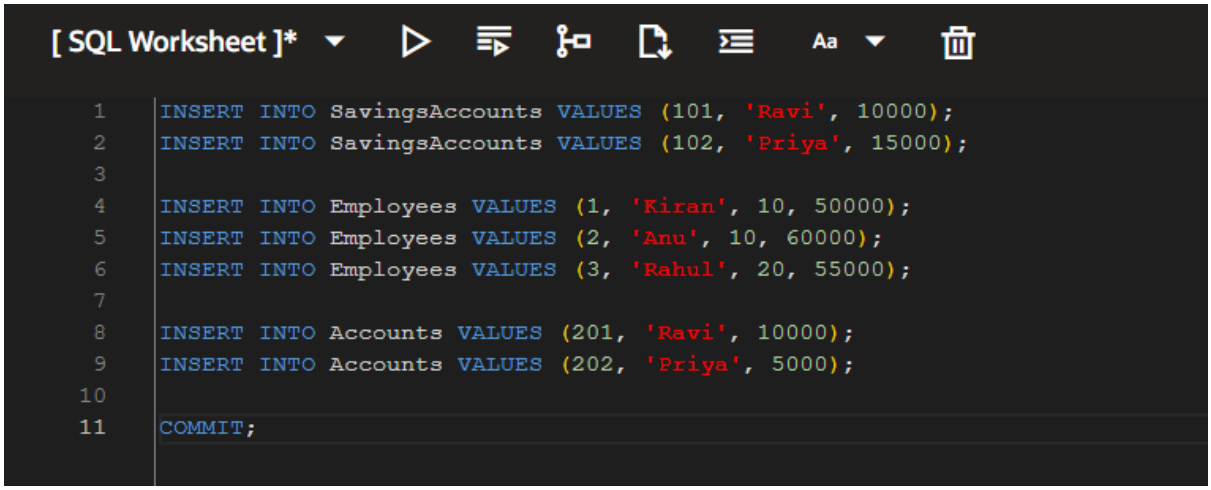
Exercise 3: Stored Procedures

Table Creation :



```
[ SQL Worksheet ]*
1  CREATE TABLE SavingsAccounts (
2      AccountID NUMBER PRIMARY KEY,
3      CustomerName VARCHAR2(50),
4      Balance NUMBER
5  );
6
7  CREATE TABLE Employees (
8      EmployeeID NUMBER PRIMARY KEY,
9      EmployeeName VARCHAR2(50),
10     DepartmentID NUMBER,
11     Salary NUMBER
12 );
13
14 CREATE TABLE Accounts (
15     AccountID NUMBER PRIMARY KEY,
16     CustomerName VARCHAR2(50),
17     Balance NUMBER
18 );
```

Insertion :



```
[ SQL Worksheet ]*
1  INSERT INTO SavingsAccounts VALUES (101, 'Ravi', 10000);
2  INSERT INTO SavingsAccounts VALUES (102, 'Priya', 15000);
3
4  INSERT INTO Employees VALUES (1, 'Kiran', 10, 50000);
5  INSERT INTO Employees VALUES (2, 'Anu', 10, 60000);
6  INSERT INTO Employees VALUES (3, 'Rahul', 20, 55000);
7
8  INSERT INTO Accounts VALUES (201, 'Ravi', 10000);
9  INSERT INTO Accounts VALUES (202, 'Priya', 5000);
10
11 COMMIT;
```

Scenario 1: The bank needs to process monthly interest for all savings accounts.

Question: Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Create Procedure

```
[ SQL Worksheet ]*
1  CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2  AS
3  BEGIN
4      UPDATE SavingsAccounts
5      SET Balance = Balance + (Balance * 0.01);
6
7      COMMIT;
8  END;
9  /
```

```
[ SQL Worksheet ]*
1  BEGIN
2      ProcessMonthlyInterest;
3  END;
4  /
5  SELECT * FROM SavingsAccounts;
```

OUTPUT:

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.007 seconds

	ACCOUNTID	CUSTOMERNAME	BALANCE
1	101	Ravi	10201
2	102	Priya	15301.5

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

Question: Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Create Procedure

```
[ SQL Worksheet ]*
1  CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
2      p_DepartmentID IN NUMBER,
3      p_BonusPercent IN NUMBER
4  )
5  AS
6  BEGIN
7      UPDATE Employees
8      SET Salary = Salary + (Salary * p_BonusPercent / 100)
9      WHERE DepartmentID = p_DepartmentID;
10
11     COMMIT;
12 END;
13 /
```

```
[ SQL Worksheet ]*
1  BEGIN
2      UpdateEmployeeBonus(10, 10);
3  END;
4  /
5  SELECT * FROM Employees;
```

OUT PUT :

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.006 seconds

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1	Kiran	10	60500
2	2	Anu	10	72600
3	3	Rahul	20	55000

Scenario 3: Customers should be able to transfer funds between their accounts.

Question: Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Create Procedure

[SQL Worksheet]*

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_FromAccount IN NUMBER,
    p_ToAccount IN NUMBER,
    p_Amount IN NUMBER
)
AS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = p_FromAccount;

    IF v_Balance >= p_Amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_Amount
        WHERE AccountID = p_FromAccount;

        UPDATE Accounts
        SET Balance = Balance + p_Amount
        WHERE AccountID = p_ToAccount;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Transfer Successful');

    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
/
```

[SQL Worksheet]*

```
BEGIN
    TransferFunds(201, 202, 2000);
END;
/
SELECT * FROM Accounts;
```

OUTPUT:

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.008 seconds

	ACCOUNTID	CUSTOMERNAME	BALANCE
1	201	Ravi	8000
2	202	Priya	7000